Blok B (80 min): Konfiguracija lokalnog okruženja

Cilj: svaki student na kraju bloka ima Linux okruženje (WSL ili nativno), konfiguriran git, prijavljen gh CLI, Python venv i prvi commit u osobnom repozitoriju u organizaciji kolegija.

Sadržaj

1. WSL i Ubuntu
2. CLI alati
3. Git
4. GitHub CLI i osobni repozitorij
5. Python i venv
6. Provjera, prvi commit i push

> macOS i Linux: korak 1 se preskače; pakete instalirati s `brew` (macOS) ili `apt` (Ubuntu). Ostali koraci su isti.

1. WSL i Ubuntu (Windows)

U PowerShellu kao administrator:

```
wsl --install -d Ubuntu-24.04
```

Ponovno pokrenuti računalo, otvoriti "Ubuntu", postaviti korisničko ime i lozinku. Zatim provjeriti:

```
wsl -l -v   # VERSION must be 2
```

U Ubuntu terminalu:

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y git curl build-essential python3 python3-venv python3-
pip
```

Savjet: projekte držati u Linux datotečnom sustavu (`~/`), ne u `/mnt/c/...` (puno sporije). VS Code: ekstenzija "WSL", pa `code .` iz terminala.

2. CLI alati

Gotovo svi alati na kolegiju (git, gh, docker, terraform, gcloud) su CLI alati i rade po istom obrascu:

```
command [subcommand [subcommand ...]] [options] [arguments]
```

Dio	Primjer	Značenje
naredba	`git`, `gh`, `docker`	program koji pokrećemo
podnaredba	`gh repo create`, `docker compose up`	radnja; može biti ugniježđena u više razina
kratka opcija	`-m "poruka"`, `-v`	jedno slovo, jedna crtica; zastavice se mogu spajati (`ls -la`)
duga opcija	`--message "poruka"`, `--private`	puna riječ, dvije crtice; čitljivija u skriptama
argument	`git add README.md`	ono nad čime se radnja izvodi

Kako se snaći s nepoznatim alatom:

```
gh --help                  # top-level commands
gh repo --help             # subcommands of "repo"
gh repo create --help      # options of one specific subcommand
git help commit            # full manual page (same as: man git-commit)
python3 --version          # almost every tool has --version
echo $?                    # exit code of the last command: 0 = success,
anything else = error
```

Konvencije koje vrijede gotovo svugdje: uglate zagrade `[ ]` u pomoći znače neobavezno, `< >` obavezno, `...` ponavljanje, a `|` izbor jedne od opcija. Izlazni kod je ono po čemu CI workflow zna je li korak uspio.

Vlastiti CLI alat u Pythonu (argparse)

Isti obrazac u 20-ak redaka, samo standardna biblioteka. Spremiti kao `greet.py`:

```
"""A tiny CLI tool that shows the command / subcommand / options
pattern."""
import argparse
import sys


def cmd_hello(args: argparse.Namespace) -> int:
    greeting = f"Hello, {args.name}!"
    print(greeting.upper() if args.shout else greeting)
    return 0


def cmd_bye(args: argparse.Namespace) -> int:
    for _ in range(args.times):
```

```
            print(f"Goodbye, {args.name}.")
        return 0


    def build_parser() -> argparse.ArgumentParser:
        parser = argparse.ArgumentParser(prog="greet", description="Greets
    people from the command line.")
        parser.add_argument("--version", action="version", version="greet
    0.0.1")
        subcommands = parser.add_subparsers(dest="command", required=True)

        hello = subcommands.add_parser("hello", help="say hello")
        hello.add_argument("name", help="who to greet")
        hello.add_argument("-s", "--shout", action="store_true", help="print in
    upper case")
        hello.set_defaults(handler=cmd_hello)

        bye = subcommands.add_parser("bye", help="say goodbye")
        bye.add_argument("name", help="who to say goodbye to")
        bye.add_argument("-n", "--times", type=int, default=1, help="how many
    times (default: 1)")
        bye.set_defaults(handler=cmd_bye)
        return parser


    def main() -> int:
        args = build_parser().parse_args()
        return args.handler(args)


    if __name__ == "__main__":
        sys.exit(main())
```

Isprobati:

```
python3 greet.py --help
python3 greet.py hello --help
python3 greet.py hello Ana --shout     # HELLO, ANA!
python3 greet.py bye Ana -n 2
python3 greet.py hello; echo $?        # missing argument -> error, exit
code 2
```

`--help` i poruke o greškama argparse generira sam. Na ovaj obrazac pišemo LLM skriptu u 2. tjednu (npr. `llm.py ask "pitanje" --model ...`).

3. Git

```
git --version
git config --global user.name "Ime Prezime"
```

```
git config --global user.email "email-s-githuba@example.com"
git config --global init.defaultBranch main
git config --global pull.rebase false
git config --list --global
```

Objasniti: što je repozitorij, radni direktorij, staging, commit (detaljno u 2. tjednu).

4. GitHub CLI i osobni repozitorij

Instalacija na Ubuntuu (službeni apt repozitorij):

```
(type -p wget >/dev/null || sudo apt install -y wget) \
 && sudo mkdir -p -m 755 /etc/apt/keyrings \
 && wget -qO- https://cli.github.com/packages/githubcli-archive-keyring.gpg
| sudo tee /etc/apt/keyrings/githubcli-archive-keyring.gpg >/dev/null \
 && sudo chmod go+r /etc/apt/keyrings/githubcli-archive-keyring.gpg \
 && echo "deb [arch=$(dpkg --print-architecture) signed-
by=/etc/apt/keyrings/githubcli-archive-keyring.gpg]
https://cli.github.com/packages stable main" | sudo tee
/etc/apt/sources.list.d/github-cli.list >/dev/null \
 && sudo apt update && sudo apt install -y gh
```

Prijava i repozitorij:

```
gh auth login            # GitHub.com → HTTPS → Login with a web browser
gh auth setup-git        # git uses gh for authentication
GITHUB_USERNAME=$(gh api user -q .login)
gh repo create unipu-praktikum-razvojne-
operacije/praktikum-$GITHUB_USERNAME --public --clone
cd praktikum-$GITHUB_USERNAME
```

Ako student nije prihvatio pozivnicu u organizaciju: `gh api user/memberships/orgs` ili prihvatiti na github.com.

5. Python i venv

```
python3 --version                # expected 3.10+
python3 -m venv .venv
source .venv/bin/activate        # prompt shows (.venv)
which python                     # must point into .venv
pip install --upgrade pip
pip install requests python-dotenv
pip freeze > requirements.txt
```

Objasniti: zašto izolirano okruženje (različite verzije paketa po projektu), zašto `.venv` **ne** ide u git, a `requirements.txt` ide. Sljedeći tjedan `requests` i `python-dotenv` koristimo za prvu LLM skriptu.

`.gitignore`:

```
cat > .gitignore <<'GI'
.venv/
__pycache__/
*.pyc
.env
GI
```

DevSecOps trenutak: `.env` će držati API ključ; nikad ne smije u repozitorij.

6. Provjera, prvi commit i push

Kopirati `check_env.sh` (dijeli ga nositelj) u repozitorij i pokrenuti:

```
bash check_env.sh | tee check_env.txt
```

Sve stavke moraju biti `OK`. Zatim:

```
echo "# Praktikum: $(gh api user -q .name)" > README.md
git add .
git status            # make sure .venv is not listed
git commit -m "chore: set up local environment"
git push -u origin main
gh repo view --web
```

(`chore:` je najava Conventional Commits iz 10. tjedna.)

Dodatak: release-please workflow

Datoteke iz bloka A dodati u repozitorij:

```
mkdir -p .github/workflows
echo "0.0.1" > VERSION.txt
# copy release_please.yml, release-please-config.json and .release-please-
manifest.json from block A
git add .
git commit -m "feat: add release-please workflow"
git push
gh run list --limit 1        # follow the run
gh pr list                   # after a few seconds: release PR
"chore(main): release 0.1.0"
```

Napomena: prije `feat` commita verzija je 0.0.1; release-please ispod 1.0.0 za `feat` podiže minor verziju (0.1.0), a za `fix` patch verziju (0.0.2).

Česti problemi

Simptom	Rješenje
`WslRegisterDistribution failed 0x80370102`	Uključiti virtualizaciju (VT-x/AMD-V) u BIOS-u
`wsl -l -v` pokazuje VERSION 1	`wsl --set-version Ubuntu-24.04 2`
`python3 -m venv` javlja grešku ensurepip	`sudo apt install python3-venv`
`gh repo create` vraća 403	Pozivnica u organizaciju nije prihvaćena ili organizacija ne dopušta članovima stvaranje repozitorija
Push traži lozinku	Pokrenuti `gh auth setup-git`
Spor rad u WSL-u	Projekt premjestiti iz `/mnt/c/...` u `~/`
release-please: "GitHub Actions is not permitted to create or approve pull requests"	U postavkama organizacije (Settings → Actions → General) uključiti "Allow GitHub Actions to create and approve pull requests"